분야 AI/SW 기초 구분 데이터베이스와 백엔드 학습시간 40시간 필수/선택 필수

정보를 깔끔하게 정리하는 디지털 서랍장 만들기

문제기술

기술적 설명



1. 미션 소개

엑셀과 DB의 차이는 '데이터가 얼마나 많냐'가 아닙니다. 테이블 사이의 '관계'를 표현할 수 있냐의 차이입니다. 그 관계를 모르면 JOIN도 ORM도 계속 헷갈립니다. 여기서 제대로 잡아두면 뒤가 편합니다. 테이블 설계부터 시작해서, 요구사항을 SQL 쿼리 하나로 해결하는 흐름을 직접 완성합니다.

데이터베이스는 서비스의 데이터를 "파일이나 엑셀"이 아니라, 관계와 규칙을 가진 형태로 안전하게 저장하고 꺼내기 위한 표준 기술입니다. 특히 SQL은 데이터 조회, 검색, 정렬, 집계 같은 실무 요구를 가장 직접적으로 해결하는 언어입니다.

이 미션에서는 백엔드 프레임워크 없이, 도메인에 맞는 테이블 구조를 설계하고(PK/FK/제약조건) 직접 데이터를 넣고(INSERT) 필요한 정보를 뽑는(SELECT/JOIN/GROUP BY) 전체 흐름을 실습합니다. 단순 쿼리 연습이 아니라 "데이터 모델링 → 데이터 입력 → 요구사항을 SQL로 해결"하는 과정을 결과물로 남깁니다.

또한, 이후 JPA/ORM을 학습할 때 필요한 관계(1:N), 키(PK/FK), 무결성, 조인 기반 조회 사고방식을 먼저 체득하게 됩니다. 즉, ORM이 해주는 일을 "SQL 관점에서" 이해할 수 있는 기반을 만드는 미션입니다.

2. 최종 결과물

다음 네 가지를 만족하는 SQL 기반 데이터베이스 실습 결과물을 완성한다. (백엔드 프레임워크 사용 금지)

1. 도메인 데이터베이스 1개 설계

- 자유 주제(예: 도서 대여, 영화 평점, 카페 주문, 여행 일정, 학급 관리 등)를 정하고 최소 4개 테이블을 만든다.
- 테이블 간 1:N 관계가 최소 2개 이상 포함되어야 한다.

2. 스키마 생성 스크립트 1개

- CREATE TABLE 로 스키마를 정의하고, PK/FK/제약조건을 포함한다.
- 실행 순서대로 정리된 .sql 파일 1개로 제출 가능해야 한다.

3. 샘플 데이터 입력 스크립트 1개

- 각 테이블에 의미 있는 샘플 데이터가 들어가도록 INSERT 를 작성한다.
- 각 테이블당 최소 10행 이상 데이터가 존재해야 한다.

4. 핵심 쿼리 15개 + 실행 결과 캡처

- 조회/조인/집계/서브쿼리/수정 및 삭제까지 포함한 쿼리 15개를 작성한다.
- 각 쿼리의 실행 결과를 스크린샷(또는 결과 텍스트)로 남긴다.

3. 과제 목표

이 과제를 마친 후, 학습자는 아래를 스스로 설명할 수 있어야 한다.

- 데이터베이스가 “엑셀과 뭐가 다른지”, 왜 테이블로 나눠 저장하는지 설명할 수 있다.
- PK/FK가 무엇이고, 1:N 관계가 데이터를 어떻게 연결하는지 말로 설명할 수 있다.
- SELECT / INSERT / UPDATE / DELETE 를 언제 쓰는지 구분할 수 있다.
- JOIN 과 GROUP BY 로 “연결된 데이터를 한 번에 뽑는 방법”을 설명할 수 있다.
- 실무에서 흔한 요구(검색/정렬/집계/랭킹)를 SQL로 어떻게 풀지 감을 잡을 수 있다.
- 인덱스가 왜 필요한지, 어떤 컬럼에 적용하면 좋은지 기초적인 이해를 할 수 있다.

4. 기능 요구 사항

다음 요구사항을 모두 만족해야 한다.

1. DB 환경 준비

- 로컬에서 실행 가능한 DB를 설치/준비한다. (예: SQLite, MySQL, PostgreSQL, H2 중 택1)
- DB에 접속해 SQL을 실행할 수 있는 도구를 준비한다. (예: DBeaver, TablePlus, DataGrip, CLI 등)
- 아래는 DB 선택 시 참고할 수 있는 가이드이다.

    - SQLite: 설치가 가장 간단하고, 파일 기반으로 별도 서버가 필요 없다. (입문자 추천)
    - MySQL: 실무에서 가장 널리 사용되며, 설치 후 서버 실행이 필요하다.
    - PostgreSQL: 표준 SQL 준수율이 높고, 고급 기능이 풍부하다.
    - H2: Java 환경에서 주로 사용하며, 인메모리 모드를 지원한다.
- DB별 문법 차이(날짜 함수, 자동 증가 키 등)가 존재하므로, 가급적 표준 SQL 범위 내에서 작성을 권장한다. DB 고유 문법을 사용한 경우, 해당 쿼리에 어떤 DB 전용 문법인지 주석으로 명시한다.

2. 데이터 모델(스키마) 설계

- 최소 4개 테이블을 설계한다.
- 각 테이블은 PK를 가진다.
- 최소 2개 이상의 FK를 사용해 1:N 관계를 만든다.
- 컬럼 타입을 의미에 맞게 선택한다. (TEXT/VARCHAR , INTEGER , DATE/DATETIME 등)
- 테이블/컬럼 이름은 역할이 드러나도록 작성한다. (member , rental , created_at 등)
- "서비스 주제(데이터 종류)"는 학습자가 직접 정하되, 최소 4개 테이블과 2개 이상의 1:N 관계를 포함할 수 있는 수준으로 선택한다. 아래는 참고할 수 있는 주제 예시이다.
    - 도서 대여: 회원, 도서, 대여 기록, 카테고리 관리
    - 영화 평점: 사용자, 영화, 리뷰, 장르 관리
    - 카페 주문: 고객, 메뉴, 주문, 주문 상세 관리
    - 온라인 쇼핑: 회원, 상품, 주문, 주문 상세 관리
    - 학급 관리: 학생, 과목, 성적, 교사 관리

3. 제약조건 적용

- 최소 1개 컬럼에 NOT NULL 을 적용한다.
- 최소 1개 컬럼에 UNIQUE 를 적용한다.
- FK가 실제로 동작하도록 설정한다. (없는 값 참조가 막혀야 한다)

4. 샘플 데이터 준비

- 각 테이블에 최소 10행 이상의 데이터를 입력한다.
- FK로 연결된 데이터가 실제로 관계를 갖도록 입력한다.

- 데이터 입력 시, FK가 참조하는 부모 테이블에 데이터가 먼저 존재해야 한다. (예: `rental` 테이블에 INSERT하기 전에 `member` 와 `book` 테이블에 데이터가 있어야 한다.)

5. 핵심 SQL 쿼리 15개 작성

- 아래 범주의 쿼리를 모두 합쳐 총 15개 이상 작성한다.
  - 기본 조회 4개 이상 (`WHERE` , `ORDER BY` , `LIMIT` 포함)
  - 조인 4개 이상 (`INNER JOIN` 2개 이상, `LEFT JOIN` 1개 이상 포함)
  - 집계 3개 이상 (`COUNT` , `SUM` , `AVG` 중 2개 이상 + `GROUP BY`)
  - 서브쿼리 1개 이상
  - 데이터 수정 및 삭제 2개 이상 (`UPDATE` , `DELETE`)
  - 인덱스 1개 이상 (`CREATE INDEX` + 적용 이유 1줄)

6. 결과 확인 자료

- 쿼리마다 실행 결과를 확인할 수 있어야 한다.
- 각 쿼리에는 “무엇을 확인하는 쿼리인지” 한 줄 설명을 붙인다.
- 결과 확인은 스크린샷 또는 결과 텍스트로 남긴다.

7. 제출물 구성

- 스키마 생성 SQL 1개 파일
- 샘플 데이터 INSERT SQL 1개 파일
- 쿼리 15개 SQL 1개 파일
- 실행 결과 캡처(이미지 또는 텍스트) 폴더 1개
- (선택) ERD 다이어그램 이미지 1개 (draw.io, dbdiagram.io 등 활용)

5. 보너스 과제

1. 조인 1개를 두 방식으로 풀기

- 같은 요구를 `JOIN` 으로도 풀고, 서브쿼리로도 풀어보고 차이를 비교해본다.

2. 데이터 정합성 깨뜨려 보기

- 일부러 FK 에러가 나는 입력을 시도하고, 왜 막히는지와 어떻게 고쳐야 하는지 기록해본다.

3. 미니 리포트 만들기

- "이 DB로 뽑을 수 있는 핵심 지표 3개"를 정의하고, 각각을 구하는 SQL을 최종본으로 정리한다. 예: 월별 대여 건수 추이, 가장 인기 있는 도서 TOP 10, 연체율이 높은 회원 목록 등

개발환경

6. 개발 환경

- 로컬에서 실행 가능한 DB를 설치/준비한다. (예: SQLite, MySQL, PostgreSQL, H2 중 택1)
- DB에 접속해 SQL을 실행할 수 있는 도구를 준비한다. (예: DBeaver, TablePlus, DataGrip, CLI 등)

제약조건

7. 제약 사항

- 백엔드 프레임워크 사용 금지: Spring/Django/Express 등으로 API나 화면을 만들지 않는다.
- DB: 로컬에서 실행 가능한 DB만 사용한다.
- 범위:
  - 뷰(View), 프로시저, 트리거 같은 고급 기능은 사용하지 않는다.
  - 정규화 이론을 과도하게 깊게 파지 않는다. 대신 “관계가 자연스럽고 쿼리가 잘 나오는 구조”를 목표로 한다.
- 제출물:
  - 스키마 생성 SQL 1개 파일
  - 샘플 데이터 INSERT SQL 1개 파일
  - 쿼리 15개 SQL 1개 파일
  - 실행 결과 캡처(이미지 또는 텍스트) 폴더 1개
  - (선택) ERD 다이어그램 이미지 1개

Test Case

8. 결과 예시

아래는 정답이 아니라 참고 예시다. 실제 주제/테이블/쿼리는 달라도 된다.

- 주제: “도서 대여”
  - 테이블 예시: `member` , `book` , `rental` , `category`
  - 관계 예시:
    - `rental.member_id` → `member.id`
    - `rental.book_id` → `book.id`
- 쿼리 예시(형태만 참고)
  - “대여 중인 책 목록”
  - “회원별 대여 횟수 집계”
  - “최근 30일 대여 기록”
  - “연체 상태로 업데이트”
  - “대여 기록이 없는 회원 찾기(서브쿼리)”